



Thread Implementation

Starting a Thread

- **Tread Function Prototype:** All thread functions have similar structure:

```
void * function_name(void * argument) {  
    // Code for thread goes here  
}
```

- The use of `void *` allows flexibility because the OS can't predict the specific data type the thread might handle
- In real-time systems, threads are often designed to run continuously, potentially looping forever
- **Thread Execution Requirements:**
 - Threads need a **starting function**, much like `main()` in C
 - Each thread must have its own **stack** for local variables and execution history
 - Threads need a **context**, which includes the instruction pointer, stack pointer and other data for proper execution

Managing Thread Context

- **Context:** A thread's context includes:
 - **Code Progress:** Keeps track of which instruction is currently being executed
 - **Register Data:** Holds the current state of the CPU registers for that thread
 - **Stack Pointer:** Points to the current location of the thread's stack
 - In larger OS setups, context may also include permission, file access, etc.

Stack Management for Threads

- Each thread must have its own **independent stack** to prevent corruption from overlapping execution traces
 - **Stack Source:** The OS allocates a portion of memory to serve as the stack for each thread
 - If the stack size is too small and a thread overflows it, data corruption can occur, particularly in systems without virtual memory
- **Stack Pool:** If thread stacks share the OS stack, there is a risk of **stack overflow**, leading to data corruption. This is the developer's responsibility to manage, especially in systems without advanced memory protections

Thread Control Block (TCB)

- A **TCB** is similar to a Process Control Block (PCB) but for threads. It tracks minimal, essential information for each thread:
 - **Identifier:** Unique thread ID
 - **Instruction Pointer:** Tracks the current execution point
 - **Stack Pointer:** Tracks the current stack location
 - Optional data might include the starting point of the thread's execution

Corruption Risks

- **Context Corruption:** If the context (like instruction pointer or stack pointer) is corrupted, the system will crash (hard fault)
- **Data Corruption:** If the thread's data is corrupted, the output may be incorrect, but the system might still run

Thread Scheduling

- **Saving and Loading Context:** For threads to share CPU time:
 - i. Save the current thread's context when it stops running
 - ii. Load the next thread's context to start its execution

- This concept, called a **context switch**, is a core part of the thread management in any OS.
- **Scheduling Decision:** Determining which thread runs next is a topic covered later, but for now, it involves saving and loading contexts as needed

Setting Up a Thread

1. **Create the TCB:** Initialize the thread's basic control data
2. **Allocate Stack Memory:** Reserve memory space for the thread's stack
3. **Setup Context:** Prepare the stack with the necessary context data:
 - Initialize specific registers (`xPSR` , `PC` , `LR` , `R3` , `R2` , `R1` , `R0`)
 - Set the Program Counter (PC) to point to the thread's execution function
 - Set the stack 16x4 bytes down from the `xPSR`

Managing threads in a Real-Time OS (RTOS)

- **TCB Management:** TCBs can be stored in lists or arrays, depending on the OS's design
- **Kernel Stack Pointer:** The OS itself uses the **Main Stack Pointer (MSP)**
- **Tracking Threads:** The OS must keep track of how many threads exist, which one is running, and available stack space

Basic RTOS Functionality

- **Kernel Initialization:** Set up kernel variables, initialize chipset settings, create TCBs, and allocate stack pointers
- **Creating a thread:** Allocate stack space, initialize the thread stack, and update the OS variables
- **Starting Threads:** The OS must handle the execution of the first thread and manage switching between threads
- **Key Functions:**
 - `kernelInit()` : Initialize the kernel
 - `osCreateThread()` : Create and register a new thread

- `kernelStart()` : Start the thread
- `osSched()` : Run the scheduler to decide which thread executes next
- `osYield()` : A thread voluntarily gives up CPU control to allow the scheduler to run

Terminating Threads

- IN simple OS implementations, an ending thread can cause a system crash if not handled properly
 - **Proper Termination:** Requires deallocating resources and removing the thread from the scheduler to prevent memory leaks
 - In complex systems like UNIX, these processes are managed carefully to avoid resource leaks